

INFORME PC2

Detección concurrente de fraude en transacciones PaySim1

Programación Concurrente y Distribuida · 1ACC0065

Docente: Herminio Paucar Curasma · NRC: 8646

INTEGRANTES

Avalos Sánchez, César Gabriel (u202310307)
Rojas Cuadros, Fabian Marcelo (u202218498)
Ballon Villar, Diego Eduardo (u201520327)
Cuadrado Jimenez, Sebastian Antonio (u202312882)

Ciclo académico 202620

Índice

1. Resumen del trabajo	3
2. Objetivos del trabajo	3
3. Investigación bibliográfica	4
4. Análisis del caso de uso	13
5. Limpieza y preprocesamiento de PaySim1	15
6. Repositorio e historial Git	19
7. PC2: modelo, implementación y evaluación	20
8. Referencias	23

1. Resumen del trabajo

El presente trabajo forma parte de la Práctica Calificada 1 (PC1) del curso de Programación Concurrente y Distribuida, y tiene como objetivo investigar cómo la programación concurrente y paralela potencia el rendimiento de modelos de machine learning al enfrentar grandes volúmenes de datos. Para ello, el grupo seleccionó y analizó cuatro papers académicos publicados en los últimos años (2023-2025), obtenidos de fuentes indexadas como Cambridge Data & Policy, IEEE Xplore, MDPI y arXiv, accedidos a través del Centro de Información de la UPC.

Los papers investigados abordan distintos casos de uso de clasificación mediante machine learning: (1) la priorización de supervisión en contratación pública mediante modelos de regresión logística y Random Forest (VigIA), (2) la detección de fraude financiero en transacciones digitales utilizando algoritmos de gradient boosting como XGBoost y LightGBM, (3) la aceleración del entrenamiento de modelos de ensamble (Random Forest, AdaBoost, XGBoost, KNN) mediante procesamiento paralelo en entornos multicore, y (4) el análisis del consumo energético asociado a distintos paradigmas de paralelización (memoria compartida, memoria distribuida y GPU) en el entrenamiento de modelos de ML.

A partir de este análisis bibliográfico, el grupo seleccionó como caso de uso propio la detección de fraude en transacciones financieras, tomando como fuente el dataset PaySim1 (Synthetic Financial Datasets For Fraud Detection, Kaggle), en línea con el paper investigado por César Gabriel Avalos Sánchez ("Financial Fraud Detection using Machine Learning Models"), que aborda directamente esta problemática utilizando el mismo dataset. Sobre este dataset, compuesto por más de 6.3 millones de registros de transacciones simuladas, se aplicó un procedimiento de limpieza y preprocesamiento que incluyó tratamiento de variables identificadoras, tratamiento de valores atípicos, codificación de variables categóricas, ingeniería de variables sobre los saldos de las cuentas y balanceo de clases. Todo el desarrollo del trabajo, incluyendo el historial de commits de cada integrante, fue gestionado a través de un repositorio de GitHub siguiendo la metodología Git Flow.

2. Objetivos del trabajo

Objetivo general:

Investigar y analizar el uso de la programación concurrente y distribuida en modelos de machine learning, evaluando los beneficios que esta aporta en términos de eficiencia computacional, tiempos de entrenamiento e inferencia, mediante la revisión de literatura académica reciente y la aplicación práctica sobre un dataset de gran volumen (>1,000,000 de registros).

Objetivos específicos:

- Identificar y analizar un paper académicos por integrante del grupo (2021-2025), publicados en fuentes indexadas reconocidas (Scopus, IEEE Xplore, ACM Digital Library, entre otras), relacionados con el uso de concurrencia en modelos de machine learning.
- Describir los algoritmos, frameworks, bibliotecas y patrones de concurrencia empleados en cada paper investigado, incluyendo su pseudocódigo cuando corresponda.
- Emitir una opinión crítica y fundamentada sobre los beneficios y/o perjuicios que aporta la programación concurrente en cada uno de los casos investigados.
- Seleccionar y sustentar un caso de uso propio, justificando la elección de un dataset con más de un millón de registros.
- Aplicar un procedimiento de limpieza y preprocesamiento de datos sobre el dataset seleccionado, documentando cada paso realizado.
- Gestionar el desarrollo colaborativo del trabajo mediante un repositorio de GitHub, evidenciando el historial de commits individual de cada integrante del grupo.

3. Explicación de cada tema investigado

Alumno	Diego Eduardo Ballon Villar	César Gabriel Avalos Sánchez	Fabian Marcelo Rojas Cuadros	Sebastian Cuadrado
Título del paper	VigIA: Prioritizing Public Procurement Oversight with Machine Learning Models and Risk Indices	Financial Fraud Detection using Machine Learning Models	A Parallel Approach to Enhance the Performance of Supervised Machine Learning Realized in a Multicore Environment	Performance and Energy Consumption of Parallel Machine Learning Algorithms
Autores	Andrés Salazar, Juan F. Pérez, Jorge Gallego	Anita Ukwu, Shatha Ghareeb, Jamila Mustafina Kazab	Ashutosh Ghimire y Fathi Amsaad - Department of Computer Science and Engineering, Wright State University, Dayton, OH,	Xidong Wu, Preston Brazzle, Stephen Cahoon

			EE.UU.	
Año	2024	2025	2024	2023
Motivación	Los organismos de control de contratación pública tienen presupuesto y personal limitados frente a un volumen de contratos que hace inviable revisar cada proceso individualmente. El caso real: la Veeduría Distrital de Bogotá, con un presupuesto anual equivalente a apenas el 0,1% del presupuesto total de la ciudad, debe vigilar cientos de miles de contratos cada año.	El crecimiento constante de los sistemas de pago digitales y transacciones con tarjeta de crédito genera un volumen masivo de datos expuestos a ataques de fraude exigiendo modelos capaces de procesar información rápidamente.	El crecimiento en tamaño y complejidad de los datasets hace que entrenar modelos de ML consume cada vez más tiempo. Los autores buscan acelerar el entrenamiento aprovechando CPUs multicore (hardware accesible, sin necesidad de GPU), aplicándolo a un problema de clasificación real: predecir la calidad de un vino a partir de sus propiedades fisicoquímicas.	El entrenamiento de modelos de <i>Machine Learning</i> exige grandes conjuntos de datos y múltiples iteraciones computacionales. Sin embargo, la gran mayoría de las investigaciones sobre el entrenamiento concurrente se enfocan únicamente en el rendimiento, dejando de lado el consumo de energía, que es una métrica fundamental para aplicaciones de alto rendimiento.
Problema que resuelve	Cómo priorizar, entre un volumen alto de contratos, cuáles supervisar primero, cubriendo dos tipos de desperdicio a la vez: el activo (corrupción deliberada) y el pasivo (ineficiencia por falta de capacidad), que según la literatura previa representan hasta el 83% del desperdicio total en algunas entidades	Este trabajo resuelve la necesidad de comparar, optimizar y seleccionar la arquitectura de machine learning que ofrezca el mejor equilibrio entre precisión, recall y eficiencia computacional, permitiendo reducir drásticamente los falsos positivos e identificar	Clasificar la calidad de un vino (escala de 3 a 9) a partir de 11 atributos fisicoquímicos (acidez, alcohol, pH, azúcar residual, sulfatos, etc.) es una tarea de clasificación multiclase con un dataset pequeño (4,898 instancias) pero con 7 clases de salida, lo que la hace más compleja que un problema binario.	La falta de análisis y optimización sobre cómo impacta la paralelización no solo en el tiempo de ejecución computacional, sino en la eficiencia energética cuando se aplican algoritmos de <i>machine learning</i> en plataformas de alto rendimiento.

	estatales. También aborda el "problema de etiquetado selectivo": los modelos entrenados solo con casos ya detectados por la Contraloría heredan el sesgo de lo que esa entidad ya encontró.	patrones complejos de ciberataques sin comprometer los tiempos de respuesta exigidos en la industria bancaria.	El reto es lograr que el entrenamiento paralelo siga siendo eficiente incluso cuando el dataset es chico, donde el overhead de paralelizar puede no compensar la ganancia.	
Propuesta de solución	Dos componentes complementarios, ambos a nivel de contrato: (1) modelos de machine learning que predicen sobrecostos y demoras usando variables objetivas del propio proceso; (2) un índice compuesto de riesgo de irregularidad (IRIC), construido como el promedio de 11 variables binarias en tres dimensiones: falta de competencia, falta de transparencia y anomalías del proceso. Incluye una versión ponderada por el valor del contrato (IRICP).	Evaluación comparativa de algoritmos de clasificación (Logistic Regression, Naïve Bayes, Random Forest, LightGBM y XGBoost), aprovechando la escalabilidad de los modelos basados en árboles impulsados por gradiente (gradient boosting)	Aplicar la misma arquitectura paralela de ensamble (Random Forest, XGBoost, AdaBoost, KNN) al dataset de calidad de vino (Wine Quality Dataset, UCI), particionando el entrenamiento entre los cores disponibles, y comparando el resultado contra el mismo experimento con el dataset de fraude (mucho más grande) para observar cómo cambia el beneficio de la concurrencia según el tamaño del dataset.	Implementar varios algoritmos clásicos de aprendizaje automático bajo diferentes paradigmas de paralelización (memoria compartida, memoria distribuida y aceleración por GPU) para medir empíricamente cómo la complejidad del modelo y el tamaño de los datos de entrenamiento afectan la eficiencia paralela y la energía consumida.
Algoritmos usados y	Algoritmo A: Cálculo del Índice de Riesgo de Irregularidad en Contratación (IRIC) Entrada:	Algoritmo: Pipeline de detección de fraude con balanceo SMOTE Entrada: Dataset	Algoritmo 1: Bootstrap Sampling para generar subconjuntos Entrada: Dataset	Algorithm 1 (del paper): Stochastic Gradient Descent (SGD) Entrada: Número de épocas de

Pseudocódigo	Registro de contrato c, historial del proveedor Salida: IRIC(c) e IRICP(c) 1. Para cada contrato c: 2. Calcular 11 variables binarias (0/1) en 3 dimensiones: 3. Falta de competencia (5 variables): proponente único, proveedor multipropósito, historial excepcionalmente alto del proveedor, contratación directa, régimen especial, período de aviso extremo 4. Falta de transparencia (2 variables): errores o datos faltantes, período de decisión extremo 5. Anomalías del proceso (3 variables): proveedor con sobrecostos previos, proveedor con retrasos previos, ausencia de proceso de contratación 6. IRIC(c) = promedio aritmético de las 11 variables binarias 7. peso(c) = $\log(\text{valor_contrato}(c)) / \text{percentil_75}(\log(\text{valor}), \text{tipo_contrato})$	de transacciones (100,000 registros, 1% fraude) Salida: Modelo entrenado y evaluado 1. Cargar dataset y verificar ausencia de nulos/duplicados 2. Codificar variables categóricas (TransactionType, Location, MerchantID) mediante label encoding 3. Normalizar variables numéricas (transaction amount) con MinMaxScaler 4. Aplicar SMOTE (Synthetic Minority Over-Sampling Technique): 5. Para cada instancia de la clase minoritaria (fraude): 6. Generar muestras sintéticas interpolando entre vecinos cercanos de la misma clase 7. Balancear el dataset con las muestras sintéticas generadas 8. Dividir dataset en 80% entrenamiento / 20% prueba	original T (tamaño N), número de subconjuntos M Salida: Lista de subconjuntos S 1. Inicializar lista vacía S 2. Para i = 1 hasta M: 3. Inicializar subconjunto vacío subset_i 4. Mientras tamaño(subset_i) < N: 5. Seleccionar aleatoriamente una instancia de T (con reemplazo) 6. Agregar la instancia a subset_i 7. Agregar subset_i a S 8. Retornar S Algoritmo: Entrenamiento paralelo del ensamble (Random Forest) 1. Particionar datos de entrenamiento con Bootstrap Sampling (Algoritmo 1) 2. Para cada subconjunto (en paralelo, uno por core): 3. Construir un árbol de decisión (CART) sobre ese subconjunto 4. Seleccionar subconjunto aleatorio de features en cada split	entrenamiento T, tasa de aprendizaje η, tamaño de mini-batch b Inicializar: Parámetros del modelo x0 1. Para t = 1, 2, ..., T: 2. Extraer muestras de mini-batch Bt = $\{\xi^j\}$, j=1..b, con Bt = b, del dataset de entrenamiento D 3. $x_t = x_{t-1} - \eta \nabla_x f(x_{t-1}; B_t)$ 4. Retornar xT Paralelización síncrona (shared-memory, con OpenMP): 5. Usar "#pragma omp parallel for" para acelerar la multiplicación de vectores en cada iteración 6. No paralelizar por batch-size, para evitar condiciones de carrera en la actualización del modelo Paralelización asíncrona (shared-memory): 7. Usar "#pragma omp parallel for schedule(static,1)" para paralelizar cada iteración de actualización 8. Usar "#pragma omp critical" para forzar que los hilos actualicen el
----------------------------	--	--	---	--

	8. $IRICP(c) = IRIC(c) \times \text{peso}(c)$ 9. Retornar $IRIC(c)$, $IRICP(c)$ Algoritmo B: Modelo de predicción de ineficiencias (sobrecostos/retrasos) Entrada: Dataset de contratos con variables del proceso Salida: Modelo entrenado + variables más relevantes 1. Balancear clases mediante remuestreo (igual número de contratos con y sin sobrecostos/retrasos) 2. Estandarizar variables explicativas 3. Entrenar Random Forest sobre todas las variables disponibles 4. Calcular importancia de características (feature importance) por árbol 5. Seleccionar subconjunto reducido de variables más importantes (ej. 7 variables para sobrecostos, 6 para retrasos) 6. Entrenar modelo final con Regresión Logística y Random Forest usando el	(muestreo estratificado para mantener proporción fraude/no-fraude) 9. Para cada modelo en [Regresión Logística, Naive Bayes, Random Forest, XGBoost, LightGBM]: 10. Aplicar GridSearchCV con validación cruzada estratificada de 5 folds para optimizar hiperparámetros 11. Entrenar modelo con datos balanceados 12. Evaluar con exactitud, precisión, recall, F1-score y ROC-AUC 13. Comparar los 5 modelos (+ Autoencoder como baseline de detección de anomalías) 14. Seleccionar mejor modelo (Random Forest: 99% exactitud, 0.99 recall, AUC=1.00)	5. Combinar todos los árboles entrenados (Ensemble) 6. Predicción final = voto mayoritario de todos los árboles	modelo uno a la vez (evitar condiciones de carrera) Paralelización distribuida (MPI, síncrona): 9. Cada nodo cliente muestrea una porción de los datos y calcula su propio gradiente local 10. Un servidor central agrega los gradientes de todos los clientes 11. El servidor actualiza el modelo y lo redistribuye a todos los nodos Paralelización distribuida (MPI, asíncrona): 12. Al recibir un gradiente de un nodo, el servidor actualiza el modelo inmediatamente y lo devuelve solo a ese nodo (sin sincronización global)
--	--	---	---	---

	subconjunto reducido (70% train / 30% test) 7. Calcular gráficos de dependencia parcial por variable 8. Evaluar con exactitud, precisión, recall, curva ROC y AUC 9. Retornar modelo con mejor AUC (Random Forest, AUC=0.943 vs 0.821 de Regresión Logística)			
Descripción de servicios, framework, bibliotecas usadas, código, lenguaje de programación, patrones.	Regresión logística y Random Forest como modelos de clasificación (elegidos explícitamente por su interpretabilidad frente a redes neuronales, que se descartan por su opacidad). Selección de variables mediante la importancia de las características del Random Forest. Gráficos de dependencia parcial para explicar cada predictor. Métricas de evaluación: exactitud, precisión, exhaustividad, curva ROC y AUC. Fuente de datos: la plataforma SECOP II de Colombia Compra Eficiente, cruzada con el registro de	Uso de pandas y Numpy para la carga, filtrado y transformación matricial y preprocesamiento de vectores de características en memoria. Implementación de la biblioteca scikit-learn para la construcción de los modelos base (logistic regression, naive bayes, random forest), la codificación de variables categóricas, el escalamiento de datos y la división de conjuntos de entrenamiento y prueba. Utilización de XGboost y LightGBM para la paralelización multihilo nativa a nivel del cpu	Python 3 (en los 3 dispositivos de prueba). Preprocesamiento: PCA (Análisis de Componentes Principales) para reducir dimensionalidad del dataset de fraude (ya venía aplicado en el dataset original de Kaggle) y estandarización (media 0, desviación 1) en ambos datasets; split 80/20 train/test. Patrón de diseño: paralelismo de datos vía data partitioning + bootstrap sampling, con sincronización entre procesos/cores para agregar resultados (patrón map-reduce a	Implementaciones desarrolladas en los lenguajes de programación C++ (para regresión logística y algoritmos genéticos) y Python (para redes neuronales). Utiliza patrones de paralelismo de memoria compartida, paralelismo de memoria distribuida y delegación de procesamiento mediante aceleración por GPU.

	sanciones de la Superintendencia de Industria y Comercio, ambas de datos.gov.co. No especifican el lenguaje de programación en el cuerpo del artículo, pero el stack de modelos (regresión logística, random forest, curvas ROC) es estándar en Python con scikit-learn o en R con caret/randomForest.	durante la búsqueda de divisiones óptimas de los árboles y el cálculo de matrices de hessiana y gradiente. Aplicación de métricas para conjuntos desbalanceados mediante scikit-learn.metrics, incluyendo precisión, recall, f1 y análisis de curvas ROC y AUC. Synthetic Financial Datasets For Fraud Detection (PaySim1) https://www.kaggle.com/datasets/ealaxi/paysim1	nivel conceptual: particionar -> procesar en paralelo -> combinar/votar). Código fuente disponible en GitHub: https://github.com/aashutoshghimire/parallel-ensemble-model. Datasets: Kaggle (Fraud Detection Credit Card) y UCI ML Repository (Wine Quality).	
Opinión crítica de su parte (estudiante) de los beneficios o perjuicios del uso de la programación concurrente.	Fabian: El paper no menciona concurrencia de forma explícita, pero el uso de Random Forest ya trae ese potencial implícito: cada árbol del ensamble puede entrenarse de forma independiente, algo que confirma el paper de Multicore que yo investigué. El	Diego: Este paper sí es más explícito que el mío en el uso de concurrencia: XGBoost y LightGBM paralelizan de forma nativa el cálculo de gradientes y la búsqueda de splits en los árboles. El beneficio es claro para datos	Diego: Este es el paper más riguroso del grupo en términos de medir concurrencia: cuantifica el speedup real por número de cores (hasta 9.05x para Random Forest). Comparado con VigIA, que ni siquiera discute el tema, aquí queda	Diego: Este paper aporta algo que ningún otro paper del grupo considera: el consumo energético, no solo la velocidad. Frente a VigIA, que ni siquiera piensa en escalabilidad, este trabajo obliga a preguntarse si vale la pena

	perjuicio es que, al priorizar interpretabilidad sobre rendimiento (descartan redes neuronales por opacidad), los autores no exploraron si ese mismo Random Forest podría escalar a nivel nacional sin rediseñar el pipeline para procesamiento paralelo. César: Comparado con mi paper de Financial Fraud, donde XGBoost y LightGBM sí discuten su paralelismo interno, VigIA se queda corto en ese aspecto: usan Random Forest pero nunca hablan de cuántos núcleos o qué tiempo de entrenamiento les tomó procesar los datos de SECOP II. Dado que manejan "cientos de miles de contratos", el beneficio de paralelizar sería real, pero el paper simplemente no lo cuantifica, a diferencia de otros papers del grupo que sí miden el speedup. Sebastian: Desde la óptica de mi	financieros masivos en tiempo real. Sin embargo, no explican cómo manejan la sincronización entre hilos cuando los datos están tan desbalanceados (fraude vs. no fraude), lo cual podría generar particiones de trabajo muy desiguales entre cores. Fabian: Comparado con el paper de Multicore que yo trabajé, que sí mide el speedup exacto por número de cores, este paper de fraude se queda en un nivel más superficial: menciona que XGBoost y LightGBM paralelizan internamente, pero no reporta tiempos de entrenamiento ni cuántos hilos usaron. Es un beneficio "de caja negra", confían en que la librería ya resuelve la concurrencia, sin analizar si esa paralelización interna es óptima	clarísimo que no todos los algoritmos se benefician igual del paralelismo, KNN apenas mejora, mientras que los basados en árboles casi escalan linealmente. El perjuicio es que todo el experimento se queda en un solo nodo multicore, sin explorar si las conclusiones se sostendrían en un clúster distribuido como el que necesitaría VigIA a escala nacional. César: A diferencia de mi paper de fraude financiero, que asume la paralelización de XGBoost como una caja negra, este paper sí abre esa caja: explica el bootstrap sampling, la construcción paralela de árboles y la combinación por votación. El beneficio es una comprensión mucho más profunda del "cómo". La limitación que veo es que el dataset de fraude que usan (284,807	paralelizar un modelo si el ahorro de tiempo no compensa el gasto energético adicional, una pregunta que sería muy relevante aplicar al caso de contratación pública si algún día se lleva a producción a gran escala. César: Comparado con mi paper de fraude, que usa paralelismo interno de librerías (XGBoost/LightGBM) sin cuestionarlo, este paper analiza explícitamente distintos paradigmas (memoria compartida, memoria distribuida, GPU), lo cual es mucho más completo. El beneficio es que da un panorama más realista de las opciones de concurrencia disponibles. El perjuicio es que, al combinar C++ para regresión logística/algoritmos genéticos y Python para redes neuronales, la comparación
--	--	--	---	---

	paper (que mide consumo energético del paralelismo), noto que VigIA tampoco discute el costo computacional de calcular los gráficos de dependencia parcial para cada predictor, algo que sí podría beneficiarse de cómputo paralelo si se aplica a los cientos de miles de contratos mencionados. Es un vacío compartido con otros papers del grupo: se preocupan por la exactitud del modelo, pero ignoran completamente el costo energético o de tiempo de escalarlo.	para su caso de uso específico. Sebastian: Desde la perspectiva energética que trabajé en mi paper, este es el caso donde más se nota la ausencia de esa métrica: paralelizar árboles de gradiente sobre "volumenes masivos de datos financieros en tiempo real" tiene un costo energético que ni siquiera se menciona. El perjuicio que señalan (gestión cuidadosa de memoria para evitar cuellos de botella) va en la línea correcta, pero se queda corto al no conectar ese cuello de botella con el consumo energético que representa.	registros) es mucho menor al de mi paper, así que sus conclusiones sobre "datasets grandes" podrían no sostenerse con volúmenes de datos verdaderamente masivos (millones de registros). Sebastian: Este paper comparte con el mío el interés en medir el paralelismo de forma empírica, pero se queda solo en tiempo de ejecución, sin tocar el consumo energético que sí exploro yo. Sería interesante cruzar ambos hallazgos: si Random Forest escala casi linealmente en velocidad con más cores, ¿escala también linealmente en consumo de energía, o hay ahí también rendimientos decrecientes? El paper no da pistas sobre eso, y es una oportunidad perdida para complementar su análisis de "punto de quiebre" del paralelismo	de consumo energético entre lenguajes podría estar sesgada por las diferencias inherentes de eficiencia entre ambos, no solo por el paradigma de paralelismo usado. Fabian: Comparado con el paper de Multicore que yo investigué, que mide speedup pero ignora energía, este paper llena ese vacío de forma directa. El beneficio es evidente: en aplicaciones de alto rendimiento, el consumo energético es tan crítico como el tiempo de cómputo, algo que mi propio paper no consideró. El perjuicio que noto es que no comparan sus resultados de energía contra los mismos algoritmos (Random Forest, XGBoost) que usa el paper de Multicore, lo cual hubiera permitido una comparación directa entre "cuánto se gana
--	--	---	--	--

				en tiempo" y "cuánto se gasta en energía" para los mismos modelos.
Link de acceso al paper.	https://www.cambridge.org/core/journals/data-and-policy/article/vigia-prioritizing-public-procurement-oversight-with-machine-learning-models-and-risk-indices/34D04747A94A7099E3CD8B91221338ED	https://ieeexplore.ieee.org/document/11367919	https://www.mdpi.com/2504-4990/6/3/90	https://arxiv.org/abs/2305.00798

4. Análisis del caso de uso.

- Contexto y problema abordado

El estudio parte de un problema del sector financiero: el crecimiento sostenido de los sistemas de pago digitales y las transacciones con tarjeta genera un volumen masivo de datos expuestos a fraude, lo que exige modelos capaces de procesar información rápidamente y con alta precisión. El desafío no es solo detectar fraude, sino hacerlo minimizando los falsos positivos —que generan fricción innecesaria al usuario— sin sacrificar los tiempos de respuesta que exige la industria bancaria..

- Justificación de la elección del dataset: Synthetic Financial Datasets For Fraud Detection (PaySim1)

La elección de PaySim1 como dataset propio del equipo responde a tres criterios. Primero, cumple con holgura el requisito de volumen del curso: sus 6.3 millones de registros son varias veces superiores al mínimo de un millón exigido, lo que garantiza que las técnicas de procesamiento concurrente y distribuido evaluadas tengan un impacto medible en tiempos de entrenamiento. Segundo, mantiene coherencia directa con la revisión bibliográfica del grupo, ya que reproduce el mismo dataset utilizado en el paper de Ukwu et al. (2025) analizado por César Avalos, lo que permite contrastar los resultados propios con los reportados en la literatura y evaluar si las afirmaciones del paper sobre paralelización nativa en XGBoost/LightGBM se sostienen empíricamente. Tercero, su fuerte desbalance de clases (apenas 0.13% de transacciones fraudulentas) y su estructura transaccional con componente temporal (**step**) lo convierten en un caso de uso representativo de los desafíos reales de detección de fraude en sistemas

financieros a gran escala, exigiendo no solo modelos precisos sino también eficientes en su entrenamiento e inferencia.

- Objetivo y enfoque técnico

El objetivo es comparar, optimizar y seleccionar la arquitectura de machine learning que ofrezca el mejor equilibrio entre precisión, recall y eficiencia computacional. Para esto se evalúan cinco algoritmos de clasificación —Regresión Logística, Naive Bayes, Random Forest, XGBoost y LightGBM— sobre el dataset sintético PaySim1 (Synthetic Financial Dataset For Fraud Detection).

El pipeline sigue un flujo estándar: carga y preprocesamiento con pandas/NumPy, codificación de variables categóricas y escalamiento, división 80/20 en entrenamiento y prueba, entrenamiento de cada modelo y evaluación final con precisión, recall, F1 y curva ROC/AUC.

El componente de concurrencia aparece de forma específica en XGBoost y LightGBM, que aprovechan multihilo nativo a nivel de CPU durante dos operaciones puntuales del boosting: el cálculo en paralelo del gradiente y la hessiana de la función de pérdida en cada nodo del árbol, y la búsqueda paralela del mejor split entre hilos. A diferencia de un esquema de bagging (como Random Forest), donde los árboles se construyen de forma independiente, aquí los árboles se combinan secuencialmente (boosting), por lo que la paralelización ocurre dentro de la construcción de cada árbol, no entre árboles.

- Resultados relevantes

El paper reporta métricas de calidad del modelo (precisión, recall, F1, ROC-AUC) para comparar los cinco algoritmos y así "seleccionar el modelo con mejor equilibrio precision/recall/eficiencia". Sin embargo, no se reportan tiempos de entrenamiento, número de hilos usados, ni ningún tipo de medición de speedup asociado a la paralelización de XGBoost/LightGBM. La ganancia de eficiencia computacional se afirma pero no se cuantifica empíricamente.

- Aporte del trabajo

El valor del paper está en la comparación sistemática de algoritmos clásicos y de gradient boosting para un caso de uso financiero real (PaySim1), evaluados bajo métricas adecuadas para datos desbalanceados (recall, F1, AUC en lugar de solo accuracy). El uso de XGBoost y LightGBM se justifica precisamente por su escalabilidad

y su paralelización nativa a nivel de CPU, lo cual los hace atractivos para "volumenes masivos de datos financieros en tiempo real".

- Limitaciones

Paralelismo de "caja negra": el paper menciona que XGBoost y LightGBM paralelizan internamente el cálculo de gradiente/hessiana y la búsqueda de splits, pero no analiza si esa paralelización es óptima para su caso de uso específico, ni reporta cuántos hilos se usaron ni el tiempo de entrenamiento resultante. Se confía en que la librería resuelve la concurrencia sin verificarlo empíricamente.

Sincronización bajo desbalance de clases: no se explica cómo se maneja la sincronización entre hilos cuando los datos están fuertemente desbalanceados (fraude vs. no fraude), lo que podría generar particiones de trabajo desiguales entre cores durante la búsqueda de splits.

Ausencia de métrica energética: el paper menciona la necesidad de una "gestión cuidadosa de memoria para evitar cuellos de botella", pero no conecta ese cuello de botella con el consumo energético que implica paralelizar árboles de gradiente sobre datos masivos en tiempo real.

Sin comparación de tiempos entre modelos: al no reportar tiempos de entrenamiento para ningún algoritmo (ni siquiera para los que no usan paralelismo, como Regresión Logística o Naive Bayes), no es posible establecer una relación costo-beneficio real entre la ganancia en velocidad que ofrece la paralelización nativa y la mejora (o no) en las métricas de clasificación.

5. Explicación del procedimiento de limpieza del dataset.

- Descripción general del conjunto de datos

El conjunto de datos seleccionado y procesado corresponde a PaySim1 (Synthetic Financial Datasets For Fraud Detection), disponible públicamente en la plataforma Kaggle. Este dataset fue generado a partir de un simulador basado en registros transaccionales reales extraídos de un servicio móvil de dinero financiero, desarrollado para investigar la detección de fraudes ante la escasez de datos financieros abiertos por motivos de confidencialidad.

El conjunto unificado se compone de 6,362,620 transacciones y 11 variables (10 predictoras y 1 objetivo), cumpliendo holgadamente con el requisito mínimo de 1,000,000 de registros exigido en la rúbrica del curso. La variable objetivo, isFraud, es de naturaleza binaria (1 = transacción fraudulenta, 0 = transacción legítima). El conjunto presenta un severo desbalance de clases, donde únicamente 8,213 transacciones son fraudulentas, representando aproximadamente el 0.1291% del total

de observaciones (frente a 6,354,407 transacciones legítimas, equivalentes al 99.8709%).

URL Dataset: <https://www.kaggle.com/datasets/ealaxi/paysim1>

URL Notebook de Limpieza en GitHub (Rama nuevo-dataset):

https://github.com/GimZtanCool/TB-Programacion_concurrente/blob/nuevo-dataset/limpieza_paysim_fraud.ipynb

- Integridad y diagnóstico de valores faltantes y duplicados

Se realizó un diagnóstico exhaustivo de integridad sobre las 6,362,620 filas:

Valores faltantes: Se verificó que ninguna de las 11 columnas presenta valores nulos o vacíos (0% de nulos en la totalidad del conjunto). La naturaleza sintética y calibrada del simulador PaySim garantiza la completitud de los registros transaccionales.

Registros duplicados: Se comprobó la inexistencia de filas completamente duplicadas (0 registros duplicados íntegros), garantizando que cada fila corresponde a un evento transaccional único en el tiempo.

- Tratamiento de variables identificadoras y regla heurística

El dataset original incluye variables alfanuméricas identificadoras de cuentas y una regla fija tradicional:

- nameOrig (cuenta de origen) y nameDest (cuenta de destino): poseen cardinalidad masiva prácticamente individual (millones de identificadores únicos tipo C1231006815 o M1979787155). Utilizar estas columnas directamente causaría un severo sobreajuste (overfitting) e incrementaría la dimensionalidad a millones de categorías en memoria.
- Extracción semántica: Se identificó que el prefijo del destinatario codifica el tipo de entidad receptora: C (Cliente particular / Customer) y M (Comercio / Merchant). Se extrajo la variable categórica binaria dest_type (C vs. M) y posteriormente se eliminaron las columnas de identificadores originales.
- Evaluación y descarte de isFlaggedFraud: Esta variable simula el sistema de control bancario tradicional basado en la regla fija de marcar transferencias individuales superiores a \$200,000 USD. El análisis exploratorio reveló que de los 8,213 fraudes reales, la regla fija apenas marcó 16 transacciones (omitiendo 8,197 fraudes, con una tasa de falsos negativos del 99.80%). Por constituir una regla heurística ineficaz y redundante frente a los modelos de machine learning, fue eliminada de la matriz de entrenamiento.

- Tratamiento de valores atípicos en montos (amount)

La variable amount presentó una marcada asimetría hacia la derecha (cola pesada). Mientras que la media general se sitúa en \$179,861.90 USD, el monto promedio de las transacciones fraudulentas alcanza los \$1,467,967 USD, frente a \$178,197 USD en transacciones legítimas. Mediante el análisis de percentiles se determinó que el 99.99% de las observaciones se encuentra por debajo de \$9,615,000 USD, con valores máximos extremos que superan los \$92.4 millones de USD. Con el fin de evitar inestabilidades numéricas en el cálculo de gradientes durante el entrenamiento, se aplicó un criterio de recorte (winsorizing) acotando los montos superiores al percentil 99.99.

- Ingeniería de características contables y temporales

A partir del análisis del dominio financiero y de la literatura especializada, se diseñaron cuatro variables predictoras de alto impacto:

- Discrepancia contable de origen (errorBalanceOrig): $\text{newbalanceOrig} + \text{amount} - \text{oldbalanceOrig}$. En transacciones legítimas esta discrepancia es nula o mínima, mientras que en transacciones fraudulentas los defraudadores suelen vaciar la cuenta a cero sin que coincida con el monto exacto.
- Discrepancia contable de destino (errorBalanceDest): $\text{oldbalanceDest} + \text{amount} - \text{newbalanceDest}$. Captura anomalías donde la cuenta receptora no refleja el ingreso completo o presentaba saldo previo inexistente.
- Variables temporales cíclicas (hora_del_dia y dia_de_la_semana): La variable original step representa horas acumuladas continuas (1 paso = 1 hora, abarcando 744 pasos correspondientes a un mes completo de 31 días). Se derivaron $\text{hora_del_dia} = \text{step} \% 24$ y $\text{dia_de_la_semana} = (\text{step} // 24) \% 7$, capturando patrones circadianos y estacionales propios de la actividad criminal digital.

- Codificación de variables categóricas

Las variables categóricas nominales del dataset corresponden a type (con 5 modalidades: CASH_IN, CASH_OUT, DEBIT, PAYMENT, TRANSFER) y dest_type (C, M). El análisis exploratorio demostró que el fraude se concentra única y exclusivamente en los tipos TRANSFER (4,097 fraudes) y CASH_OUT (4,116 fraudes), siendo nulo en las restantes. Ambas variables fueron codificadas mediante One-Hot Encoding (pd.get_dummies con drop_first=True), transformándolas en columnas binarias con tipo numérico entero de bajo consumo de memoria (int8).

- Balance de clases mediante ponderación

Dado que la clase minoritaria (fraude) representa únicamente el 0.1291% del dataset masivo de 6.36 millones de registros, el uso de técnicas de sobremuestreo sintético como SMOTE multiplicaría innecesariamente el volumen de datos en memoria a más de 12.7 millones de filas, colapsando los recursos durante el entrenamiento concurrente en Go previsto para la PC2. En su lugar, se implementó una estrategia de ponderación de clases (class weighting) en la función de pérdida con pesos balanceados calculados analíticamente: $W0 \approx 0.5006$ para legítimas y $W1 \approx 387.35$ para fraudes, penalizando severamente los falsos negativos sin generar sobrecarga computacional.

- División del conjunto de datos (Split temporal)

Debido a la naturaleza estrictamente secuencial y temporal de las transacciones financieras respecto de la variable step, la división del dataset en subconjuntos de entrenamiento y prueba se efectuó siguiendo un criterio cronológico (80% entrenamiento / 20% prueba): 5,090,096 transacciones iniciales para entrenamiento (5,651 fraudes) y 1,272,524 transacciones cronológicas posteriores para prueba (2,562 fraudes). Este procedimiento previene la fuga de información temporal (data leakage), asegurando que los modelos evalúen transacciones futuras sin haber visto datos posteriores en su entrenamiento.

- Estandarización

Las variables numéricas continuas y las derivadas de saldos y tiempos fueron estandarizadas con media cero y desviación estándar uno ($z = (x - \mu) / \sigma$). El escalador (StandardScaler) fue ajustado exclusivamente sobre el conjunto de entrenamiento (80%) y aplicado tanto al conjunto de entrenamiento como al de prueba, evitando cualquier filtración de información estadística hacia el conjunto de evaluación.

Tabla 1.

Resumen del procedimiento de limpieza y preprocesamiento

Etapas	Técnica aplicada	Justificación
Diagnóstico de calidad	Verificación de completitud y duplicados	Dataset sintético consistente: 0 nulos y 0 duplicados íntegros en 6,362,620 filas
Tratamiento de identificadores	Depuración de nameOrig y nameDest; extracción de prefijo dest_type	Prevenir sobreajuste por IDs individuales y capturar la distinción Cliente vs Comercio

Depuración de regla estática	Eliminación de isFlaggedFraud	Regla basada en umbral fijo (>200k) ineficaz (99.8% falsos negativos; 16 de 8,213 fraudes)
Valores atípicos (amount)	Recorte (winsorizing) al percentil 99.99 (\$9,615,000 USD)	Controlar colas pesadas y asegurar la estabilidad numérica en el cálculo de gradientes
Ingeniería contable	Derivación de errorBalanceOrig y errorBalanceDest	Identificar discrepancias de saldo contable típicas en operaciones ilícitas
Ingeniería temporal	Derivación cíclica de hora_del_dia (step % 24) y dia_de_la_semana	Capturar estacionalidad diaria y semanal sin depender del contador lineal acumulado
Variables categóricas	One-Hot Encoding (type, dest_type) con drop_first=True	Representación binaria compacta (int8) compatible con modelos de aprendizaje automático
Balance de clases	Ponderación de clases en función de pérdida ($W_0 \approx 0.5006$, $W_1 \approx 387.35$)	Compensar el desbalance (0.13% fraude) sin duplicar memoria RAM ni sobrecargar Go
División del conjunto	Split cronológico 80/20 por variable step	Simular entorno de producción real y evitar fuga de información temporal (data leakage)
Estandarización	Z-score (StandardScaler) ajustado solo sobre entrenamiento	Escalamiento uniforme ($\mu=0$, $\sigma=1$) indispensable para convergencia y algoritmos métricos

6. Trabajo en github, historial de commits de cada integrante del grupo.

URL del repositorio: https://github.com/GimZtanCool/TB-Programacion_concurrente

Rama oficial del entregable PC1: nuevo-dataset

(https://github.com/GimZtanCool/TB-Programacion_concurrente/tree/nuevo-dataset)

El desarrollo colaborativo se estructuró siguiendo la metodología Git Flow:

- Rama main: Contiene la configuración base inicial y la documentación del proyecto.
- Rama nuevo-dataset: Rama de trabajo donde se integró el dataset masivo PaySim1 (6,362,620 registros) gestionado mediante Git LFS (paysim.csv), junto con el notebook

CC65 PC2 - Detección de fraude en PaySim1

1. Correcciones de PC1 y alcance

Este entregable conserva el análisis bibliográfico, el caso de uso y el procedimiento de limpieza de PC1 y añade el modelado concurrente, la implementación en Go y la evaluación solicitada para PC2. Se usó `paysim.csv`, el CSV original completo con 6,362,620 transacciones; los archivos de data - limpia/ son muestras, no la matriz limpia completa.

Al recalcular el pipeline sobre el CSV se corrigieron tres cifras publicadas en PC1:

Dato	Cifra anterior	Resultado reproducible
Percentil 99.99 de amount	USD 9,615,000	USD 22,919,326.54
Fraudes en entrenamiento cronológico (80%)	5,651	3,959
Fraudes en prueba cronológica (20%)	2,562	4,254
Predictores después de codificación	14	15

El dataset mantiene 8,213 fraudes en total. La división respeta el orden cronológico del CSV: 5,090,096 registros para entrenamiento y 1,272,524 para prueba. La estandarización usa media y desviación poblacional calculadas solo con entrenamiento; el umbral de recorte se obtiene con el percentil interpolado 99.99 del conjunto completo, como en el flujo previo de limpieza.

2. Modelo y características

Se implementó regresión logística binaria con descenso por gradiente batch y ponderación de clases. Para una fila con vector de características x , etiqueta y y parámetros w, b , la probabilidad estimada es $p = \text{sigmoid}(w \cdot x + b)$. El error se pondera con 0.5006 para la clase legítima y 387.35 para la clase fraude, y se promedia sobre las filas de entrenamiento. Se usa tasa de aprendizaje 0.1 y regularización L2 de 0.0001 sobre los pesos. El sesgo no se regulariza.

Las 15 características, en orden, son `step`, `amount`, los cuatro saldos originales, `errorBalanceOrig`, `errorBalanceDest`, `hora_del_dia`, `dia_de_la_semana`, cuatro indicadores de `type` (`CASH_OUT`, `DEBIT`, `PAYMENT`, `TRANSFER`) y `dest_type_M`. Las columnas de identificadores y la bandera heurística `isFlaggedFraud` se excluyen. El código calcula el percentil, deriva características, estima el escalador y prepara las particiones directamente desde el CSV completo.

3. Implementación secuencial y worker pool

Ambas variantes ejecutan la misma actualización del modelo y usan las mismas filas y parámetros. La versión secuencial acumula gradiente de cada fila en una sola goroutine. La versión concurrente divide el rango de filas en segmentos contiguos y los envía por un canal de trabajos a un conjunto fijo de workers. Cada worker calcula un gradiente parcial privado y lo envía por un canal con capacidad suficiente. La goroutine principal recibe todos los parciales, los reduce en un único gradiente y actualiza los pesos una vez por época.

Pseudocódigo de la versión concurrente:

```
inicializar pesos y sesgo
repetir por época:
    crear canal de trabajos y canal de resultados
    iniciar W workers
    enviar a cada worker un rango disjunto de filas
    cada worker calcula y publica su gradiente parcial local
    cerrar trabajos; esperar a los workers; cerrar resultados
    reducir resultados en el coordinador
    actualizar pesos y sesgo una sola vez
```

La exclusión de escrituras compartidas se obtiene por diseño: ningún worker modifica los pesos ni el gradiente de otro; solo el coordinador reduce y aplica la actualización después de recibir los resultados. Se usan goroutines,

canales y `sync.WaitGroup` de la biblioteca estándar de Go. No se requiere `Mutex` porque no hay estado mutable compartido entre workers.

4. Modelo Promela

`pc2/sync.pml` abstrae dos workers y un coordinador. Cada worker marca su parcial privado como listo y envía su identificador por un canal acotado. El coordinador verifica que el parcial exista y no haya sido consumido, registra cada resultado una sola vez y actualiza el estado compartido una sola vez después de recibir ambos resultados. Las aserciones comprueban que se consumieron los dos parciales y que el número de actualizaciones es exactamente uno.

La estructura del modelo cubre las intercalaciones de envío y recepción sin acceso concurrente de los workers al estado del modelo. El ejecutable `Spin` no está instalado en el entorno disponible (tampoco hay gestor de paquetes en WSL), por lo que el análisis exhaustivo del archivo `Promela` no pudo ejecutarse aquí. La verificación formal con `Spin` queda pendiente; no se presenta como resultado ya comprobado.

5. Medición de rendimiento

Se midió una pasada completa de gradiente sobre las 5,090,096 filas de entrenamiento. Se ejecutaron 30 repeticiones por configuración; se descartaron los tres tiempos menores y los tres mayores y se promediaron los 24 restantes (media recortada al 10%). La medición cronometra el cómputo, la coordinación de workers, la reducción y la actualización; excluye lectura del CSV, cálculo del percentil y estandarización. El entorno reportó Windows x64, Go 1.27.0 y 12 procesadores lógicos.

Configuración	Media recortada (ms)	Speedup	Eficiencia sobre N workers
Secuencial	263.355	1.000x	-
Worker pool, 2	135.617	1.942x	97.1%
Worker pool, 4	69.375	3.796x	94.9%
Worker pool, 8	39.880	6.604x	82.6%

$\text{Speedup} = \text{tiempo secuencial} / \text{tiempo concurrente}$; la eficiencia es $\text{speedup} / \text{workers}$. La latencia bajó de 263.355 ms a 39.880 ms al pasar a ocho workers. El crecimiento casi lineal indica que este cálculo batch se beneficia de dividir filas y reducir solo 15 acumuladores por worker. La eficiencia menor al 100% refleja coordinación, reducción y variaciones del sistema; las mediciones no implican que otros tamaños de datos o máquinas obtengan la misma curva.

6. Recursos, calidad del modelo y trade-offs

Durante la medición el proceso empleó en promedio 0.2594, 0.2620, 0.2667 y 0.3016 segundos de CPU para las configuraciones secuencial, 2, 4 y 8 workers respectivamente. El heap asignado observado después de cargar las matrices fue de aproximadamente 712 MiB. La CPU usada crece levemente al paralelizar por la coordinación y ejecución de varias goroutines, mientras que el tiempo de pared cae de manera marcada. El punto de equilibrio práctico de esta carga medida se alcanza con dos workers; cuatro y ocho siguen mejorando la latencia en el equipo usado.

Como comprobación funcional, se entrenó cinco épocas con la versión secuencial y con la concurrente usando 12 workers. Ambas dieron la misma matriz de confusión en prueba: 3,731 verdaderos positivos, 285,109 falsos positivos y 523 falsos negativos. Con umbral 0.5, esto equivale a recall 87.71%, precisión 1.29% y F1 2.55%. La ponderación favorece la detección de fraudes, pero genera muchos falsos positivos; se necesita selección de umbral y evaluación de precisión-recall antes de cualquier uso operativo. Esta evaluación funcional no se mezcló con los tiempos de una época del benchmark.

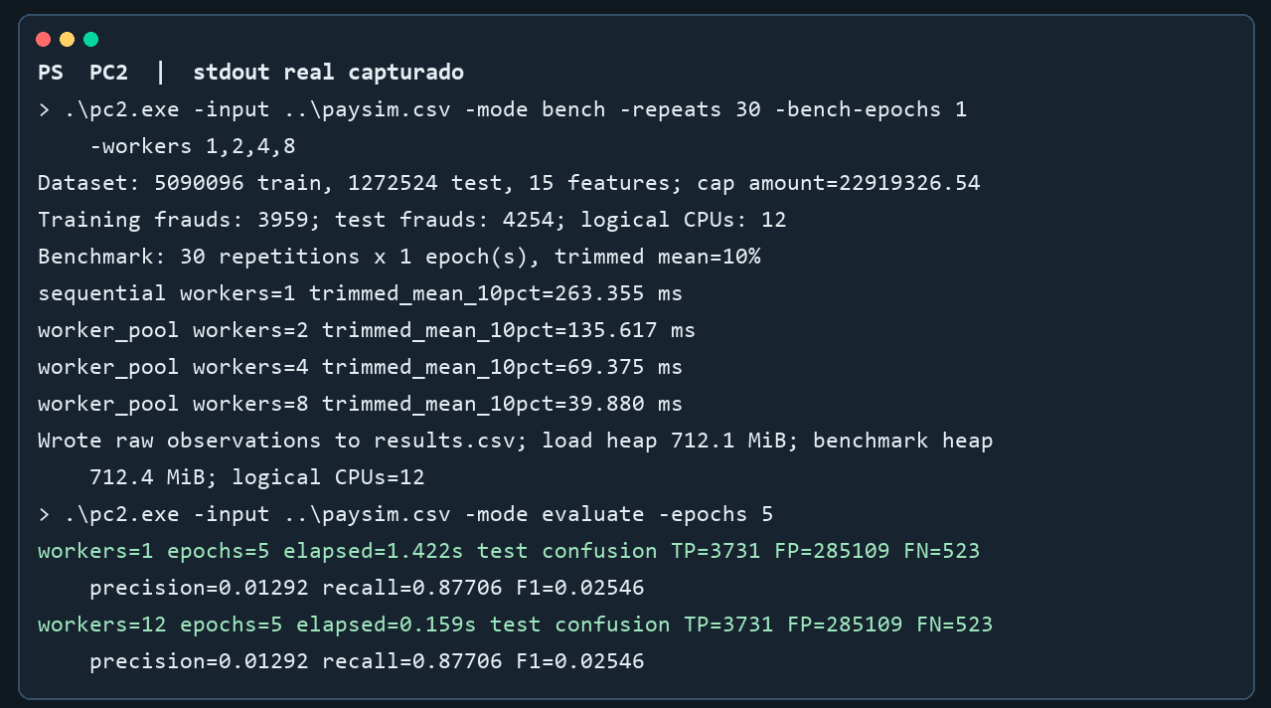
El speedup medido corresponde al núcleo de gradiente sobre datos ya residentes en memoria. La preparación del dataset requiere tres pasadas por el CSV y no se incluye en esa métrica; por ello el speedup de extremo a extremo será menor. La carga es limitada por lectura de memoria y cálculo de sigmoid, y aumentar workers también aumenta el uso simultáneo de CPU. Para datasets pequeños, la creación y coordinación de workers puede costar más que el trabajo paralelo.

7. Reproducibilidad y evidencias

Desde la carpeta pc2/:

```
go run . -input ../paysim.csv -mode bench -repeats 30 -bench-epochs 1 -workers 1,2,4,8 -out results.csv
go run . -input ../paysim.csv -mode evaluate -epochs 5
```

Las observaciones individuales están en pc2/results.csv. El código fuente es pc2/main.go; el modelo está en pc2/sync.pm1; la guía está en pc2/README.md. La figura siguiente se generó directamente desde la salida stdout capturada del programa; el CSV conserva cada repetición para comprobar los promedios. Las limitaciones principales son la falta de verificación Spin ejecutada y que solo se midió una pasada de gradiente a cuatro niveles de paralelismo en una máquina.



```
PS PC2 | stdout real capturado
> .\pc2.exe -input ../paysim.csv -mode bench -repeats 30 -bench-epochs 1
-workers 1,2,4,8
Dataset: 5090096 train, 1272524 test, 15 features; cap amount=22919326.54
Training frauds: 3959; test frauds: 4254; logical CPUs: 12
Benchmark: 30 repetitions x 1 epoch(s), trimmed mean=10%
sequential workers=1 trimmed_mean_10pct=263.355 ms
worker_pool workers=2 trimmed_mean_10pct=135.617 ms
worker_pool workers=4 trimmed_mean_10pct=69.375 ms
worker_pool workers=8 trimmed_mean_10pct=39.880 ms
Wrote raw observations to results.csv; load heap 712.1 MiB; benchmark heap
712.4 MiB; logical CPUs=12
> .\pc2.exe -input ../paysim.csv -mode evaluate -epochs 5
workers=1 epochs=5 elapsed=1.422s test confusion TP=3731 FP=285109 FN=523
precision=0.01292 recall=0.87706 F1=0.02546
workers=12 epochs=5 elapsed=0.159s test confusion TP=3731 FP=285109 FN=523
precision=0.01292 recall=0.87706 F1=0.02546
```

Evidencia visual generada desde la salida stdout de las ejecuciones

oficial de preprocesamiento (limpieza_paysim_fraud.ipynb), los scripts de partición y las muestras de validación (data-limpia/).

The screenshot shows a GitHub repository page for 'TB-Programacion_concurrente'. The repository is public and has 3 branches and 0 tags. The main branch is selected. The file list shows several files: 'data-limpia', '.gitattributes', '.gitignore', 'README.md', 'limpieza_paysim_fraud.ipynb', and 'paysim.csv'. The README file is open, showing the title 'Detección de Fraude en Transacciones Financieras Masivas (PaySim1)' and a description of the project. The right sidebar shows the repository's activity, including stars, forks, and releases.

Repository: TB-Programacion_concurrente (Public)

3 Branches, 0 Tags

Go to file, Add file, Code

Files:

File	Commit Message	Time Ago
data-limpia	feat: migración a nuevo dataset PaySim1 (>6.3M registros), pi...	40 minutes ago
.gitattributes	feat: migración a nuevo dataset PaySim1 (>6.3M registros), pi...	40 minutes ago
.gitignore	chore: remover archivos CC65 del repositorio e ignorar en .git...	24 minutes ago
README.md	Fix formatting and headings in README.md	now
limpieza_paysim_fraud.ipynb	feat: migración a nuevo dataset PaySim1 (>6.3M registros), pi...	40 minutes ago
paysim.csv	feat: migración a nuevo dataset PaySim1 (>6.3M registros), pi...	40 minutes ago

README

Detección de Fraude en Transacciones Financieras Masivas (PaySim1)

CC65 - Programación Concurrente y Distribuida | Trabajo Parcial 2026-20

Repositorio del proyecto del curso Programación Concurrente y Distribuida (CC65), enfocado en el procesamiento masivo de datos, análisis exploratorio (EDA) y preparación de un conjunto de datos a gran escala para el entrenamiento e inferencia de modelos de Machine Learning mediante arquitecturas concurrentes y paralelas (Go, Goroutines, Worker Pools y verificación formal en Promela/Spin).

About

No description, website, or topics provided.

Readme, Activity, 0 stars, 0 watching, 0 forks, Report repository

Releases

No releases published. [Create a new release](#)

Packages

No packages published. [Publish your first package](#)

Contributors (4)

- SebasUPC Sebastian
- DiegoEBV Nobody
- garra0507 César Avalos
- GimZtanCool

7. 8. Referencias.

Salazar, A., Pérez, J. F., & Gallego, J. (2024). VigIA: Prioritizing public procurement oversight with machine learning models and risk indices. *Data & Policy*, 6, e75. <https://doi.org/10.1017/dap.2024.83>

Ukwu, A., Ghareeb, S., & Kazan, J. M. (2025). Financial fraud detection using machine learning models. In *2025 18th International Conference on Development in eSystem Engineering (DeSE)* (pp. 435-439). IEEE. <https://doi.org/10.1109/DeSE68208.2025.11367919>

Ghimire, A., & Amsaad, F. (2024). A parallel approach to enhance the performance of supervised machine learning realized in a multicore environment. *Machine Learning and Knowledge Extraction*, 6(3), 1840-1856. <https://doi.org/10.3390/make6030090>

Wu, X., Brazzle, P., & Cahoon, S. (2023). *Performance and energy consumption of parallel machine learning algorithms* (arXiv:2305.00798). arXiv. <https://arxiv.org/abs/2305.00798>